ENGINE ↻ ENGINE · LOSSLESS SEED V1 · CAPTURED 2026-08-26

Depth Engine Meta-Run

The depth engine, improved by a depth-engine run. Operator's opening order: *“make sure it's lossless in the sense that I don't miss a single idea.”* Every idea from the voice transcript is on this page under a stable ID; the verbatim transcript is preserved separately as ground truth. If an idea is missing here, that is a defect — flag it.

8	7	6	10	10	12
FAILURES	MECHANISMS	STANDING RULES	TARGETS · ALL VERIFIED	CLAIMS CHECKED	DECODES

4
GATES ON YOU

§8 BLOCKED ON THE OPERATOR

read this first

G1 **Run `! codex login`.** Verified still logged out on 2026-08-26. Until it's in, every verdict in this run — including the breaker subagents you asked for — is same-family, and same-model whenever Fable caps. For a run where the model judges changes to its own harness, that's the worst possible place to be weak. Bonus: logging in also lets us re-run the retained smb-site-rescue verification prompts through a second brain.

G2 **Confirm one decode: "just had CN" = shadcn.** The feature story was custom components in TypeScript/JSX being switched over. Everything else in the decode log is certain; this is the only real ambiguity.

G3 **Point me at the canonical Matt Pocock channel.** He publishes on YouTube, X, and AI Hero. The yt-brain pipeline needs one channel URL/handle to scrape all videos (likely `@mattpocockuk` on YouTube — confirm).

G4 **Scale ruling on the landscape sweep.** You said "literally everything out there" on agents, harness engineering, loop engineering, context management. Your own bulk-scope rule says exhaustive rarely means literal. Recommended: ranked sweep — top ~30 repos by relevance, then loop until two consecutive rounds surface nothing new. Confirm, or order the literal crawl.

§1 OBSERVED FAILURES

lived evidence, operator-reported

F1 No self-checkpoint, no "good enough — stop here"

During the TypeScript/JSX component feature, the engine burned tokens repeatedly fixing its own documentation after the audit was already done — 128 components, researched, switchable-vs-not already answered. No agent ever rules "this is sufficient for implementation." It ground through every gate until the operator manually opened the audit and read it himself.

> "the whole point of the depth engine is to try to automate everything."

F2 Thoroughness is not adaptive

The meticulousness is by design — robust, nothing left unturned — and is not itself the flaw. The flaw: the engine can't tell when full research-and-breadth mode is warranted versus when to just get the work done. It needs judgment, not a lobotomy.

F3 Stale and dying subagents go undetected

On a large feature, subagents kept dying or going stale and the engine never noticed. Reproduced on Codex, the Claude app, and the CLI — engine-level, not one harness's bug. The operator sat waiting, then checked by hand.

> "you failed as an engine."

F4 Model auto-routing not applied — and invisible

The engine supposedly routes models when spawning subagents. Inspecting the Claude Code app and `/workflows`: every single subagent was Opus 5. Nothing was routed, and there was no way to see or verify what should have been.

F5 **Diff visibility is poor**

It's hard to see what actually changed when the engine codes — the CLI and app "just kind of tell you" instead of showing line-by-line. Wants engine-level change tracking, token-efficient, no spamming. Operator explicitly has no solution in mind: this one is research.

F6 **No automatic PR/MR documentation**

Every merge request required manually telling the agent to write documentation. Real software-engineering documentation should be produced for every PR without being asked. Fix is M6.

F7 **Extreme token burn**

The engine is extremely token-intensive. That was an acceptable original trade (robustness over conservation), but efficiency management must now be built into the backend itself — optimize tokens without losing proficiency.

F8 **Raw output is unreadable "AI language"**

The engine is great at the work; the management side is not. The running chat's raw output is hard to understand, which forces the copy-paste-to-a-second-chat workaround described in M1.

§2 OPERATOR-PROPOSED MECHANISMS

`validate, then build`

M1 **A Project Manager inside the engine, with isolated judgment** `fixes F8`

Today's manual ritual: paste the engine's entire output into a fresh chat, which — from a clean, uninfluenced position — explains where the project stands, where it's going, and what to look into. That is a project manager's job, so build the PM into the engine itself.

- **Topology:** the operator talks primarily to the PM, not the engine. Direct operator↔engine access stays possible, but most traffic routes through the PM — and the PM actually *manages* the engine's work, not just narrates it.
- **The hard requirement — bias isolation, both directions:** engine output must not

shape the PM's judgment (or the PM degrades into "just another agent doing a check"), and a PM that drifts slightly wrong must not be able to steer the engine away from operator intent — even while the engine is implementing the right idea. Operator: "that needs to be figured out and instrumented well."

M2 Staleness watchdog

fixes F3

A dedicated subagent polls every other agent on an interval and reports stale/dead ones, so the user never sits waiting on a corpse. Autonomy that requires manual babysitting is disqualified.

M3 Routing that applies, and a visible model panel

fixes F4

Two halves: make auto-routing actually take effect, and show the user which subagents run on which model — Sonnet? Haiku? — with the task each cheap model is doing, so model↔task fit is verifiable. Named anti-patterns: Haiku must never reason about architecture; Fable 5 must never be spent merely reading codebase documents.

M4 Lossless humanization, built in

fixes F8

Output understandable in plain language without losing any actual value — the idea must survive intact, no lossy paraphrase. Never again "paste this elsewhere and ask it to humanize."

M5 Senior software-engineering discipline when coding

Explicitly named: DRY; TDD in the strict order (write the test, then the function, then run the function against the pre-written test); CI/CD pipelines. "Software engineering principles, not AI slop."

M6 Repo-aware automatic MR documentation

fixes F6

Every repo has its own documentation rules, and a public release must be agnostic to all of them. Procedure: first read the repo's rules (this operator's CLAUDE.md includes unbreakable ones — e.g. AI-written docs must say they're AI-written) and follow them; where no rules exist, document like a real senior engineer — no AI language, not over- or under-explained. The operator can't define that bar ("I'm not a senior engineer yet"): the engine must figure it out **and prove** the chosen standard is how senior engineers actually

document.

M7 **"Good enough" gate + thoroughness dial** `fixes F1 · F2`

Implied by F1/F2 and left open by the operator: a checkpoint where some agent can rule "sufficient for implementation, stop," plus engine-level judgment for research-mode versus get-it-done mode. Alongside M1, this is the run's core design problem.

§3 STANDING RULES

`this run and all engine work after`

R1	**Lossless.** Not a single idea dropped — ever.
R2	**Evidence, not guessing.** Technique choices (routing above all) come from published community practice and studies — "a math, STEM standpoint" — never from the model's own hunch.
R3	**Real content, every research pass.** Built-in web search bounces off Cloudflare and returns titles. Stealth scraping tooling is now mandatory on every research pass. Scrapling 0.4.15 is already installed in the run venv.
R4	**Break it before delivering it.** Subagents attack every finding and try to prove it wrong; only survivors reach the operators.
R5	**Dual mandate.** Every research output both teaches the operators what can improve the engine and is synthesized by the model into concrete engine changes.
R6	**Explanation format.** Simple English; every technical term named, then defined; audit format fit for technical decisions; every claim linked to its exact source for verification.

§4 EXTERNAL TARGETS

`all ten verified real · 2026-08-26`

T1 **Scrapling — the research fetcher** `serves R3`

Karim Shoair's adaptive scraping framework: parser relearns element positions after

redesigns; StealthyFetcher spoofs fingerprints and clears Cloudflare Turnstile; concurrent spider framework; ships an MCP server for live web pulls inside Claude at lower token cost; BSD-3. ~33k stars as of June 2026 (the video's 74k didn't check out). **Installed: v0.4.15.** Run task: wire it in as the engine's standard fetcher and survey whether anything beats it.

T2 **mattpocock/skills — the "Grill Me" target**

Not three skills — **21 composable skills**, ~200k stars, ~11M downloads, on skills.sh as `mattpocock/skills`. Matt Pocock (Total TypeScript; teaches AI coding at AI Hero). `/grill-me` interviews you in frontier-ordered rounds until every branch of a decision tree is resolved — "essentially the same idea behind the depth engine." Direct mappings to our pain: `/tdd + /code-review` ↔ M5, `/to-spec + /to-tickets` ↔ M6/PM, `/wayfinder` ↔ multi-session runs. Audit + learning task: install, dissect why they're good (or aren't), extract takeaways.

T3 **skills.sh — the registry**

Vercel's open agent-skills ecosystem (`npx skills find / add`, works across 18+ agents). Study as T2's install channel and as a distribution path if the engine ever goes public — a possibility the operator raised in M6.

T4 **Matt Pocock's channel — full audit** `gate G3`

Scrape all videos via the yt-brain pipeline; audit the whole channel; deliver per R6 — simple English, terms defined, every claim linked to its exact video. Verification also surfaced his "Sand Castle" framework (parallel sandboxed coding agents in worktrees/Docker), squarely relevant to the engine.

T5 **everything-claude-code**

Affaan Mustafa's config system — won the Anthropic × Forum Ventures hackathon building zenith.chat in 8 hours with zero hand-typed lines. Skills, hooks, instincts/memory, orchestrated multi-agent workflow; now **117k+ stars** (the video's 16k was early). Audit: what does his harness do that ours doesn't?

T6 **Meta-Harness (Stanford) — the academic version of this run**

"End-to-End Optimization of Model Harnesses" (Yoonho Lee et al., Stanford IRIS)

"End-to-End Optimization of Model Harnesses" (Yoonho Lee et al., Stanford IRIS).
Confirmed: changing the harness around a fixed LLM can swing performance **6×** on the same benchmark; prior methods over-compress feedback, destroying the execution traces needed to blame specific harness decisions — so Meta-Harness keeps full source + logs and lets an agentic outer loop iteratively rewrite the harness. Beat hand-designed harnesses incl. TerminalBench-2. Read fully; mine its method for this run's own loop.

T7 Anthropic's planner / generator / evaluator experiment

Three-agent harness vs solo agent building a 2D game maker. Verified details worth stealing: no agent may plan, code, *and* grade itself; planner specs without over-specifying implementation; generator works one-feature sprints, commits to git, self-evaluates before handoff; evaluator QAs against a sprint contract negotiated before code; feature list held as JSON, never Markdown (resists model corruption — never remove or reorder, only flip incomplete→complete); git log as permanent memory. Solo agent: broken toy, 20 min, $9. Harness: 6 h, $200, working polished app that caught subtle UI bugs.

T8 OpenAI: Harness engineering

Frontier Product Exploration team, ~5 months: internal Electron app, **1M+ lines, 1,500+ PRs, zero human-written code**, 3→7 engineers. Finding: improving the environment around the AI mattered more than improving the AI. Environment patterns from this cluster: a 100-line index instead of thousand-page manuals; isolated sandboxes; monitoring on chosen metrics instead of static-test guessing; "janitor" agents that build nothing and only enforce executable architecture rules.

T9 Claude Code internals — four production patterns

To evaluate against the engine: (1) one async-generator state machine, every event through a single unified pipeline; (2) read-only tools parallel, write tools strictly serial; (3) five context-compression tiers of escalating aggression, truncation as absolute last resort; (4) system prompt split static/dynamic so the static half caches and you pay only for new tokens. Status: plausible, unverified against source — verify during the run.

T10 The full landscape sweep — unknown unknowns

`gate G4`

Beyond named targets: many tools solve pieces of what the engine wants to solve, including problems the operators haven't identified yet, and the engine predates a fast-

moving year. Sweep everything relevant on **agents, harness engineering, loop engineering, context management**; understand each; report per R5/R6. Starting index already found: ai-boost/awesome-harness-engineering.

§5 CLAIMS LEDGER

what the quoted videos said vs. what checks out

#	CLAIM FROM TRANSCRIPT	STATUS
C1	Harness (not model) can change performance up to 6×	CONFIRMED Meta-Harness paper
C2	Meta-Harness keeps full source+logs, agent rewrites harness, beat hand-designed systems	CONFIRMED
C3	Scrapling: Cloudflare bypass, adaptive re-finding, stealth mode, MCP server, BSD-3	CONFIRMED
C4	Scrapling "74k stars", "700× faster", "2 ms parser"	UNVERIFIED NUMBERS ~33k stars; speed = vendor benchmark, test ourselves
C5	"Three skills called Grill Me"	CORRECTED one /grill-me among 21 skills
C6	ECC won Anthropic hackathon; zenith.chat in 8 h; "16k stars"	CONFIRMED stars now 117k+
C7	Anthropic 3-agent experiment: JSON checklists, git-as-memory, no self-grading	CONFIRMED
C8	OpenAI: 3 engineers, 1M-line app, zero human code; environment > model	CONFIRMED
C9	Claude Code: state machine, read-parallel/write-serial, 5 compression tiers, static/dynamic prompt	PLAUSIBLE verify against source in-run
C10	"Brilliant engineers with amnesia"; "context anxiety" → premature done-marking	FRAMING design input, not a checkable fact

§6 DECODE LOG

dictation garble → interpretation

#	TRANSCRIPT SAID		INTERPRETED AS	CONFIDENCE
D1	"d engine / Def engine / deaf engine / defendant"	→	the depth engine	CERTAIN
D2	"Typesrip and XGS … just had CN"	→	TypeScript + JSX … **shadcn**	CONFIRM — G2
D3	"scrappling, built by Kareem Shob"	→	Scrapling, by Karim Shoair	VERIFIED
D4	"Matt P Pock … three skills … Grill Me"	→	Matt Pocock; /grill-me, one of 21	VERIFIED
D5	"Skills.sH … by Verselle"	→	skills.sh, Vercel's registry	VERIFIED
D6	"Afon Mustafa … Anthropic X For Ventures … Every Claude Code"	→	Affaan Mustafa; × Forum Ventures; everything-claude-code	VERIFIED
D7	"if they can … you give it back to us"	→	if breakers **cannot** break it, deliver (R4)	HIGH
D8	"never try. Like, don't repeat yourself"	→	DRY principle (dictation stutter)	HIGH
D9	"Opus 500"	→	Opus 5	CERTAIN
D10	"loop engineering, contact management"	→	loop engineering, **context** management	CERTAIN
D11	"one small step per, such as creating a single password"	→	one small step per session — e.g. a single password *field*	HIGH
D12	"I released a free skill … testing your AI"	→	a different creator's plug inside a quoted video; sweep will catch it if it matters	NOTED